DAY 03: JAVA FOR POLYGLOTS

Data Bytes Technologies · Java 25 & Modern Stack

You already know how to program. Today we map the ideas you carry from **C#, TypeScript, Python, Ruby, or Go** onto idiomatic **Java 25**, one small step at a time, with nothing assumed.

// WHERE WE ARE: DAY 3 OF 10

The first two days were about *the workflow*. Today we meet *the language* the rest runs in.

Behind us

- **Day 1: Git**: every file we write now travels through commits, branches, review.
- **Day 2: Docker**: one command gives everyone the same reproducible environment.

Today and ahead

- **Day 3: Java for Polyglots** (you are here)
- Days 4–5: Java concurrency
- Week 2: Postgres, Kafka, and the Quarkus **app**

> 💡 Today is **not** programming basics. It's *idiomatic Java 25*: how to **model data** and **make decisions on its shape**. That toolkit is the spine of the Order Service we build in week 2.

// BY THE END OF TODAY, YOU CAN...

- explain **how Java runs** (source → bytecode → JVM) and run code with `javac`, `java`, and `jshell`
- model a domain with **records** (immutable value objects) and **sealed** types
- write an **exhaustive** `switch` that destructures records, no `default` needed
- reach for the **standard library** the modern way: text blocks, Streams, `Optional`, `java.time`
- tell **checked** from **unchecked** exceptions, and know Java's one-line build story

// THE FRAME FOR TODAY: A ROSETTA STONE

You're not learning to think from scratch. You're learning a **new dialect** for ideas you have.

- A *value object*: Python `@dataclass`, C# `record`, TS `readonly` type → Java **record**
- A *tagged union*: TS discriminated union, Rust `enum`, F# sum type → Java **sealed interface**
- *Branch on shape*: Python `match`, TS `switch` + `never` → Java **pattern matching**
- *Maybe-absent*: Rust `Option`, TS `T | undefined` → Java `Optional`

Every concept today gets bridged **from a language you already know**. Keep yours in mind, when I say "this is your X", that's your anchor.

// FIRST: HOW DOES JAVA ACTUALLY RUN?

If you come from Python or JS, "run my file" means an interpreter reads it. Java has **one extra step**.

> 💡 **In plain terms**: you write `.java` source; the **compiler** (`javac`) turns it into portable **bytecode**; the **JVM** runs that bytecode on any OS. The JVM is a managed runtime. It has a **garbage collector**, just like Python, JS, Ruby, Go, and C#.

- **Bridge:** Python compiles to `.pyc` bytecode run by CPython; JS is JIT-compiled by V8. Java is the same shape (*compile then run on a VM*) just more explicit about the compile step.
- **Unlike Go**, Java doesn't produce a single native binary; it ships bytecode that the JVM runs (and **JIT-compiles** hot paths to native code while running, so it gets *faster* as it warms up).

// ONE PICTURE: SOURCE → BYTECODE → JVM

The win: the **same** `.class` runs on your laptop, a teammate's, and the production container. The JVM absorbs the OS differences. "Write once, run anywhere" is this picture.

// THREE COMMANDS YOU'LL ACTUALLY TYPE

```
javac Hello.java         # compile  → produces Hello.class (bytecode)
java  Hello              # run the class that has a main method
jar   cf app.jar *.class # package classes into one distributable archive
```

- For **learning**, skip the dance entirely. The single-file launcher compiles *and* runs in memory:

```
java Hello.java          # no .class on disk; great for experiments and the lab
```

- And `jshell` is Java's **REPL**, like `python`, `node`, or `irb`, but for Java.

// 🧪 TRY IT NOW (3 MIN): REPL AND A ONE-FILE PROGRAM

Two ways to run Java with zero setup. First, the REPL:

```
jshell> "hello".toUpperCase()
jshell> var x = 21 * 2
```

Then a whole program in one file, note: **no class wrapper, no** `public static void main` :

```
printf 'void main() { IO.println("hi from a one-file program"); }\n' > Hi.java
java Hi.java
```

That stripped-down `void main()` is **JDK 25's compact source file** (final, no flags), Java's new soft landing, as light as a Python script.

// `VAR`: TYPE INFERENCE, NOT DYNAMIC TYPING

Java is still **statically typed**. `var` just means "infer the type from the right-hand side."

```
var price = new Money(1299, "USD");     // inferred: Money
var names = List.of("ada", "linus");    // inferred: List<String>
var total = 0L;                          // inferred: long
```

> 💡 **In plain terms**: `var` is **C#'s** `var` / **TS's** `let` **with inference**, *not* Python's dynamic `x`. The type is fixed at compile time; you just didn't have to spell it out.

Use it when the type is obvious from the right side. Spell the type out when it adds clarity. `var` is for readability, not laziness.

// NOW THE FUN PART: MODELING DATA

This is the real reason we're here. Two features do most of the work in a modern domain model:

- **Records**: short, immutable *value objects*. One line of code, full behaviour.
- **Sealed types**: a *closed set* of permitted shapes the compiler knows about.

Master these two and you can describe almost any business concept precisely, and let the **compiler** check you handled it. Let's take them one at a time.

// RECORDS: WHY THEY EXIST

In older Java, a simple "bag of values" cost ~40 lines: fields, a constructor, getters, `equals`, `hashCode`, `toString`. People got it wrong, and the boilerplate hid bugs.

> 💡 **In plain terms**: a **record** is a transparent, immutable value object. You name the data once; Java generates the constructor, accessors, `equals`, `hashCode`, and `toString` for you.

If you've written a C# `record`, a Python `@dataclass(frozen=True)`, a Kotlin `data class`, or a TS `readonly` type, this is Java's version of the same idea.

// RECORDS: THE SAME VALUE OBJECT, FIVE LANGUAGES

```go
// Go
type Money struct {
    Cents    int64
    Currency string
}
```

Same idea everywhere: name the fields, get an immutable value. Java's one line gives you accessors (`money.cents()`), value equality, and a readable `toString` for free.

// RECORDS AREN'T JUST DATA: ADD BEHAVIOUR AND GUARDS

A record can have methods, and a **compact constructor** that validates on the way in.

```
public record Money(long cents, String currency) {
    public Money {                          // compact constructor, runs before fields are set
        if (cents < 0) throw new IllegalArgumentException("cents must be >= 0");
    }
    public Money plus(Money other) {        // behaviour lives with the data
        return new Money(cents + other.cents, currency);
    }
}
```

Validate once, in the constructor, and the type is **always valid afterwards**. Illegal states become unrepresentable. `plus` returns a *new* `Money`; the original never mutates.

// 🧪 TRY IT NOW (3 MIN): A RECORD, LIVE

In `jshell`, define a record and watch what you get for free:

```
jshell> record Money(long cents, String currency) {}
jshell> var price = new Money(1299, "USD")
jshell> price.cents()              // generated accessor (a method!)
jshell> price                       // generated toString
jshell> new Money(1299, "USD").equals(price)   // value equality, for free
```

`price` prints `Money[cents=1299, currency=USD]`, and two `Money`s with the same values are **equal**, none of which you had to write.

// EQUALITY AND IMMUTABILITY: THE QUIET SUPERPOWER

In Java, `==` compares **identity** (same object), `equals` compares **value**. Records make `equals` / `hashCode` compare *contents* automatically.

- **Bridge:** this is Python's `==` (via `__eq__`) vs `is`, or C#'s record value-equality vs reference equality. Java's default for *classes* is identity; records flip it to value.
- Because records are **immutable** *and* have value-based `hashCode`, they're safe as **map keys** and **set members**, and safe to share across threads without locks (we lean on this Days 4–5).

```
Map<Money, Integer> stock = new HashMap<>();
stock.put(new Money(1299, "USD"), 5);
stock.get(new Money(1299, "USD"));   // 5, a *different* object, equal by value
```

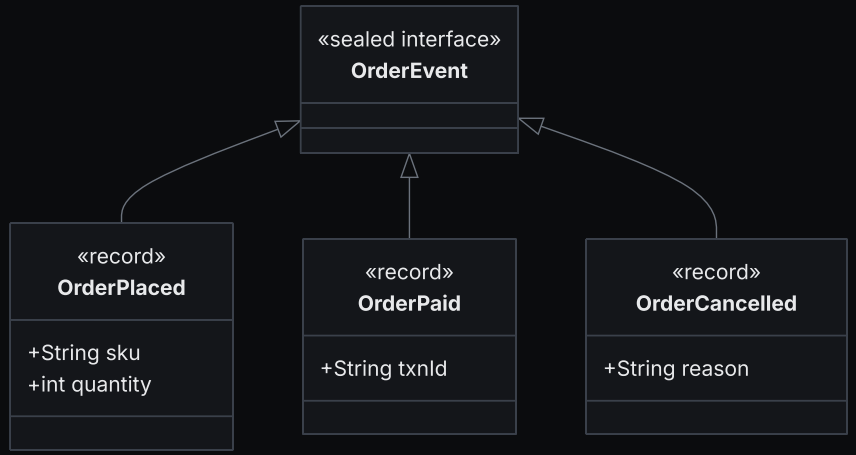
// SEALED TYPES: WHY "CLOSED" MATTERS

A record says *what one shape looks like*. A **sealed interface** says *here is the complete list of shapes*, and nothing else is allowed.

> 💡 **In plain terms**: a sealed interface defines a **closed set** of permitted types. If you've used a TypeScript *discriminated union*, a Rust `enum`, or an F# sum type, this is Java's version of exactly that.

"An order event is *exactly one of*: placed, paid, or cancelled." The compiler will hold you to that list. No rogue fourth type can sneak in.

// SEALED TYPES: ONE VISUAL



A closed, compiler-checked set of cases, the same guarantee a discriminated union gives you in TypeScript or a sum type gives you in a functional language.

// SEALED TYPES: IN CODE

```
public sealed interface OrderEvent
        permits OrderPlaced, OrderPaid, OrderCancelled {}

public record OrderPlaced(String sku, int quantity) implements OrderEvent {}
public record OrderPaid(String txnId)               implements OrderEvent {}
public record OrderCancelled(String reason)         implements OrderEvent {}
```

`permits` is the closed list. Each record `implements` the interface. Add a fourth shape later and you must add it to `permits`. The compiler keeps the list honest.

// PATTERN MATCHING: BRANCH ON THE SHAPE

Because the set is **sealed**, a `switch` over it is **exhaustive**: the compiler rejects your code if you forget a case, and you need **no** `default` branch.

Pattern matching also **destructures** records. It pulls the components straight out into variables, so you don't reach for accessors by hand.

This is the payoff that makes "records + sealed" worth it. Bridge: Python 3.10+ `match` / `case`, or TS's `switch` with a `never` exhaustiveness check. Java does both, and the compiler enforces it.

// PATTERN MATCHING: THE SWITCH

```
String describe(OrderEvent event) {
    return switch (event) {
        case OrderPlaced(var sku, var qty) -> "placed %d x %s".formatted(qty, sku);
        case OrderPaid(var txnId)          -> "paid (txn " + txnId + ")";
        case OrderCancelled(var reason)    -> "cancelled: " + reason;
    };
}
```

`OrderPlaced(var sku, var qty)` is a **record pattern**. It matches the type *and* binds `sku` and `qty` in one move. No `default`, because those three cases are all there can be.

// 🧪 TRY IT NOW (4 MIN): BUILD IT, THEN BREAK IT

In `jshell`, build the closed set and switch over it:

```
jshell> sealed interface Shape permits Circle, Square {}
jshell> record Circle(double r) implements Shape {}
jshell> record Square(double side) implements Shape {}
jshell> double area(Shape s) { return switch (s) {
   ...>     case Circle(var r)    -> Math.PI * r * r;
   ...>     case Square(var side) -> side * side;
   ...> }; }
jshell> area(new Circle(2))
```

Now **delete the `Square` case** and re-paste `area`. Watch the compiler refuse it. That refusal is the whole point.

// GUARDS: WHEN A CASE NEEDS A CONDITION

Sometimes the shape isn't enough. You want to branch on a **value** too. Add `when`:

```
return switch (event) {
    case OrderPlaced(var sku, var qty) when qty > 100 -> "bulk order of " + sku;
    case OrderPlaced(var sku, var qty)                -> "order of " + qty + " " + sku;
    case OrderPaid(var txnId)                         -> "paid: " + txnId;
    case OrderCancelled(var reason)                   -> "cancelled: " + reason;
};
```

A *guarded pattern* matches an `OrderPlaced` **and** only when `qty > 100`. Put the guarded case **before** the general one. The first match wins.

// ✋ EVERYONE WITH ME?

You should, right now, be able to:

- say what a **record** generates for you, and why two equal-valued records are `equals`
- explain what **sealed** means by the word *closed*, and name your language's equivalent
- read a **record pattern** in a `switch` and say what it binds
- explain **why no** `default` is needed, and what breaks if you add a fourth case

If any of those is fuzzy, flag it now. Text blocks, Streams, and the lab all build on this core.

// STRINGS: TEXT BLOCKS, NOT ESCAPING HELL

Multi-line strings used to mean `\n` and a forest of `\"`. A **text block** (triple quotes) keeps the text readable, and `formatted(...)` fills in the holes:

```
String json = """
    { "sku": "%s", "qty": %d }
    """.formatted(sku, qty);
```

`%s` and `%d` are placeholders; `formatted` substitutes the arguments in order, the same `printf`-style format you've seen in C, Python's `%`, or Go's `fmt`. Bridge: text blocks are Python's triple-quoted strings or JS template literals (minus interpolation).

// ⚠️ THERE ARE NO STRING TEMPLATES IN JAVA 25

> 💡 **In plain terms**: if you've read about `STR."...\{x}..."` interpolation, forget it. That preview was **withdrawn in JDK 23** and is **absent from Java 25**. It will not compile.

```
String bad  = STR."total \{amount}";          // ❌ does not exist in Java 25
String good = "total %d".formatted(amount);   // ✅ text block / formatted
```

Always reach for **text blocks + `formatted(...)`**. This one trips up people who learned from older blog posts, and the compiler error it gives is confusing.

// STREAMS: LAZY PIPELINES OVER DATA

A **stream** describes a pipeline of operations over a collection: `filter` → `map` → `collect`. Nothing runs until a **terminal** operation pulls the data through.

```
record Order(String sku, int qty) {}
var orders = List.of(new Order("A", 3), new Order("B", 0), new Order("C", 5));

int units = orders.stream()              // source
        .filter(o -> o.qty() > 0)        // intermediate, lazy
        .mapToInt(Order::qty)            // intermediate, lazy
        .sum();                          // terminal, NOW it runs → 8
```

Bridge: this is Python's comprehensions / `map` + `filter`, JS array methods, C# LINQ, Ruby's `Enumerable`. Two quirks: a stream is **lazy** (no terminal op = no work) and **single-use** (consume it once, then it's spent).

// 🧪 TRY IT NOW (3 MIN): A STREAM PIPELINE

In `jshell`, transform and group some data:

```
jshell> var words = List.of("git", "docker", "java", "kafka", "quarkus")
jshell> words.stream().filter(w -> w.length() > 4).map(String::toUpperCase).toList()
jshell> words.stream().collect(java.util.stream.Collectors.groupingBy(String::length))
```

`toList()` and `Collectors.groupingBy(...)` are **terminal**. They're what makes the pipeline actually run. Drop them and you get a `Stream` object that has done *nothing*.

// `Optional`: THE HONEST "MAYBE MISSING"

In Java, `null` is **not** in the type system: any reference can secretly be `null`, and the compiler won't warn you. `Optional<T>` makes "maybe absent" **visible** in the type.

> 💡 **In plain terms**: `Optional<T>` is Rust's `Option`, TS's `T | undefined`, or Swift's optional. Use it for **return values** that might be empty; treat raw `null` as a smell, not a tool.

```
Optional<Order> found = repository.findBySku(sku);     // might be empty
String label = found.map(Order::label)                  // transform if present
                    .orElse("unknown");                  // default if absent
```

The `.map(...).orElse(...)` chain replaces a pile of `if (x ≠ null)` checks, and you can't *forget* to handle the empty case.

// ⌨️ CODE ALONG (3 MIN): CHAIN AN OPTIONAL, NEVER TOUCH NULL

In `jshell`, find a value and supply a fallback without a single `if`:

```
jshell> var users = java.util.Map.of("ada", "admin", "linus", "kernel")
jshell> users.get("guest")                          // returns null, the old way (danger)
jshell> java.util.Optional.ofNullable(users.get("guest")).orElse("no-role")
jshell> users.keySet().stream().filter(k -> k.startsWith("a")).findFirst()
```

`findFirst()` returns an `Optional`, Java handing you "maybe one, maybe none" honestly, instead of a `null` you might forget to check.

// `JAVA.TIME`: MODEL TIME HONESTLY

> 💡 **In plain terms**: the old `java.util.Date` is dead. Pick the type that matches what you *mean*: a point on the timeline, a human wall-clock reading, or a length of time.

```
Instant now      = Instant.now();             // a point on the timeline (UTC), use for timestamps
LocalDateTime wc = LocalDateTime.now();       // a wall-clock reading, no zone
Duration ttl     = Duration.ofMinutes(5);     // a length of time
Instant expiry   = now.plus(ttl);             // arithmetic that reads like English
```

Bridge: `Instant` ≈ C#'s `DateTimeOffset.UtcNow` ≈ Go's `time.Now().UTC()` ≈ Python's timezone-aware `datetime`. Same instinct everywhere: prefer an unambiguous, zone-aware instant.

// ERRORS: JAVA'S CHECKED VS UNCHECKED EXCEPTIONS

Coming from Go's error values, Rust's `Result`, or Python/JS exceptions, Java adds one twist: some exceptions are **checked**. The compiler forces you to handle or declare them.

- **unchecked** (`RuntimeException`), *programming errors*: a bug, a broken invariant, a bad arg. You don't have to declare them. (This is what Python/JS exceptions feel like.)
- **checked** (`Exception`), *recoverable conditions at a real boundary*: a file missing, a network blip. The signature must say `throws`, and callers must deal with it.

Modern convention: throw **unchecked** for bugs, use a few **meaningful checked** types only at real boundaries, and **map** them to clean responses, exactly what the Order Service does in week 2.

// MAVEN, BRIEFLY: THE BUILD TOOL

The full `javac` / `jar` dance is automated by a **build tool**. In week 2 our app uses **Maven**, run through the `./mvnw` wrapper so everyone gets the *exact* same Maven version.

- The **POM** (`pom.xml`) declares your dependencies; a **BOM** pins a consistent set of versions for you, no hand-picking compatible numbers.
- The lifecycle phases you'll actually run: `compile`, `test`, `package`.
- Bridge: this is `npm` + `package.json`, Python's `pip` + `pyproject`, Go modules, or NuGet.

That's all you need today. For the lab, you won't even need Maven; the single-file launcher is enough.

// COMMON TRAPS

- **Thinking `var` is dynamic**: it isn't; the type is fixed at compile time, just inferred.
- **Treating a record accessor as a field**: it's `money.cents()`, a *method*, not `money.cents`.
- **Reaching for `null`**: Java won't stop you, but it's a smell. Prefer `Optional` at boundaries.
- **Adding `default` to a sealed switch**: you don't need it, and it *hides* the compiler's help when you add a new case.
- **Reusing a consumed stream**: streams are single-use; build a fresh one each time.
- **Copying `STR."..."` from a blog**: String Templates don't exist in Java 25. Use `formatted`.

// RECAP, IN PLAIN TERMS

Today you learned to **describe data** and **decide on its shape**, the Java way:

- **how Java runs**: source → bytecode → JVM, plus `jshell` and the one-file launcher
- **records**: immutable value objects with value equality, one line each
- **sealed interfaces**: a closed, compiler-checked set of shapes
- **exhaustive** `switch` with **record patterns**: branch and destructure, no `default`
- **text blocks, Streams,** `Optional`, `java.time`: the modern standard-library habits

Plus the build story (Maven, lightly) and Java's checked-vs-unchecked exceptions. That's a full modern-Java toolkit.

// TODAY'S LAB: THE ROSETTA STONE

> 🧪 **Guided lab**: take the idioms you reach for in *your* language and rewrite each as idiomatic Java 25: a value object as a **record** (with validation), a tagged union as a **sealed** type, an **exhaustive switch** over it, a **text block**, a **Stream**, and an `Optional`. Then peer-review in pairs.
>
> Brief and starter: `labs/day-03/` · run with `java starter/RosettaStone.java`, no build needed.

Acceptance: a record, a sealed interface with an exhaustive `switch`, a Stream, and one text block, all compiling, with **no** `null` as control flow and **no** `STR."..."`.

// TAKEAWAY

Modern Java models a domain as a **closed set of immutable shapes**, then lets the **compiler** check you've handled every one. Describe the data well, and correctness follows.

Tomorrow, Day 4: Java Concurrency I. We put these immutable values *in motion*. Virtual threads let you write simple, blocking, one-task-per-request code that still scales to tens of thousands of tasks. Today's records are exactly what we'll pass between them, safely.